

ALGORITHMIQUE de GRAPHE & OPTIMISATION

CH.3 : Arbre couvrant

de poids minimal

ENSI/II.1: 2022/2023

(1)

Plan du chapitre

1) Arbre couvrant à poids minimal

- Définitions, propriétés.
- Arbre Couvrant de poids Minimal (ACM).
- Algorithmes de Résolution.

ARBRE COUVRANT À POIDS MINIMAL

DÉFINITIONS :

- Un arbre est un graphe connexe sans cycle.
- Une forêt est un graphe dont chaque composante connexe est un arbre.

- THÉORÈME :

soit $T(S_T, A_T)$ un graphe d'ordre $n \geq 2$; les six propriétés suivantes sont équivalentes pour caractériser un arbre :

- 1) T est connexe et sans cycle ;
- 2) T est sans cycle et admet $n - 1$ arcs ;
- 3) T est connexe et admet $n - 1$ arcs ;
- 4) T est sans cycle et en ajoutant un arc, on crée un cycle (et un seul) ;
- 5) T est connexe, et si on supprime un arc quelconque, il n'est plus connexe ;
- 6) Tout couple de sommets est reliés par une chaîne et une seule.

Arbre Couvrant de poids Minimal (ACM)

- $G = (S, A, c)$: non-orienté, connexe et valué ($c : A \rightarrow \mathbb{R}$).
- Étant donné un graphe partiel $G' = (S, A')$ de G avec $A' \subseteq A$, on appelle poids de G' le nombre :

$$c(G') = \sum_{u \in A'} c(u)$$

- DÉFINITION :

un arbre couvrant de G est un graphe partiel, connexe et acyclique, $T(S_T, A_T)$ de G où $S_T = S$ et $A_T \subseteq A$.

- PROPOSITION :

Un graphe admet un arbre couvrant si et seulement s'il est connexe.

- Le problème de l'**arbre couvrant de poids minimum** (ACM) est de chercher un arbre T^* de G tel que:

$$c(T^*) = \min \{c(T) / T = (S, A_T) \text{ est un A.C de } G\}$$

- Le problème de l'A.C.M admet une solution car:
 - L'ensemble des arbres couvrants est fini.
 - Il admet au moins un élément de coût minimum.

Remarque 1 :

Un arbre couvrant de poids minimal est **unique** si et seulement si les poids de ces arêtes sont deux à deux distincts.

Remarque 2: on suppose dans la suite que le graphe G est connexe. S'il ne l'est pas le problème sera résolu sur chaque composante connexe de G .

- **PROPRIÉTÉ : EXISTENCE D'UN ACM**

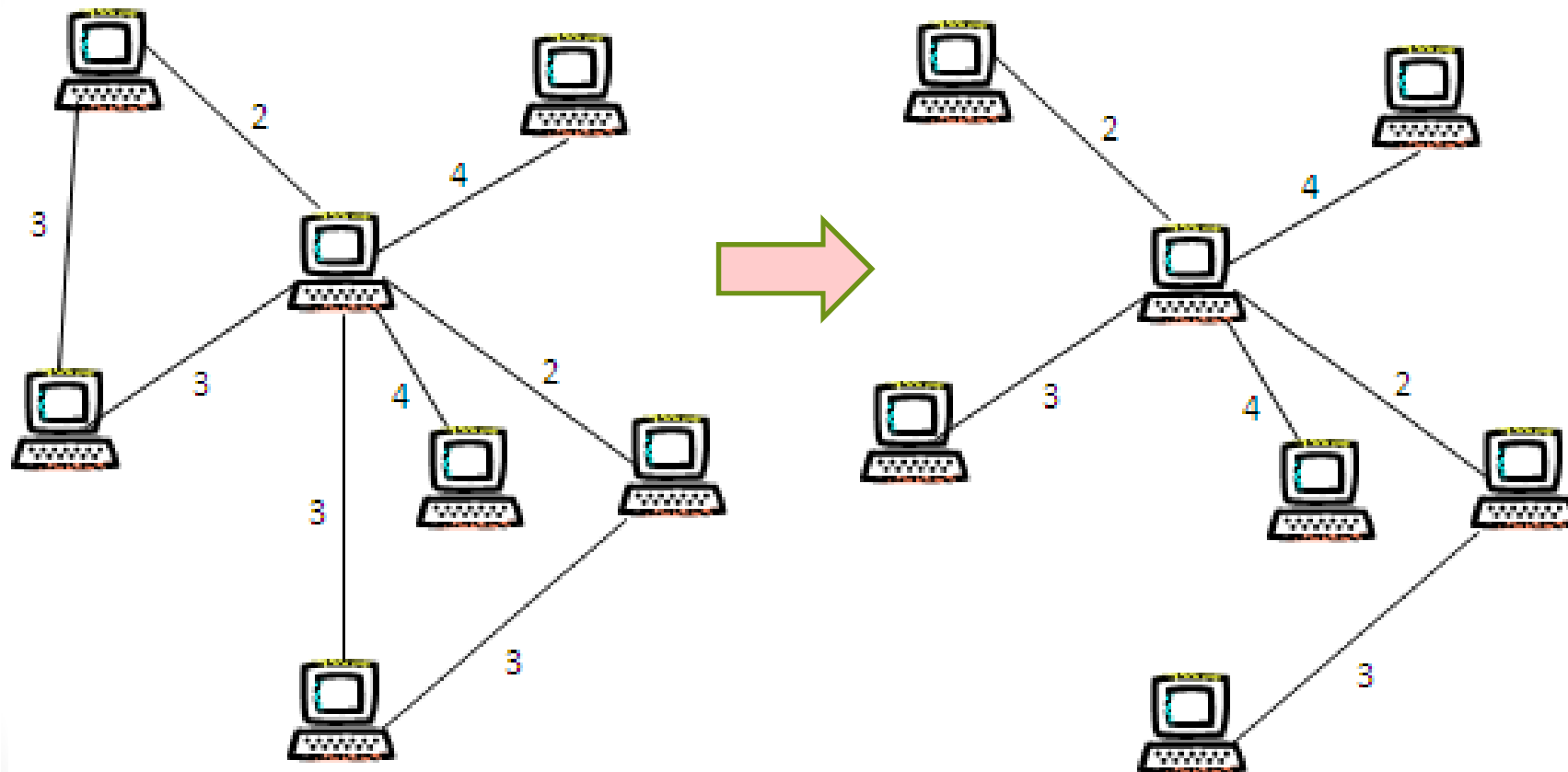
Tout graphe non-orienté, valué et connexe admet un ou plusieurs A.C.M.

Application directe : la diffusion d'information sur un réseau.

Soit G le graphe qui représente le réseau des rues d'une ville que l'on cherche à câbler de façon optimale, c-à-d: il faut trouver un graphe connexe tel que :

- Chaque sommet doit avoir accès au réseau.
- Le coût du câblage doit être minimal.

EXEMPLE D'ACM



Construction d'un ACM

- $G = (S, A, c)$: non-orienté, connexe et valué ($c : A \rightarrow \mathbb{R}$).
- On va construire un A.C.M ; $T(S_T, A_T)$ de manière incrémentale.
 - Au début, A_T est vide.
 - A chaque étape, on va choisir une nouvelle arête (x, y) tq
 $A_T \cup \{(x, y)\}$ est toujours un sous-ensemble d'un A.C.M pour G .

Le problème : Comment connecter à moindre coût des sommets non encore reliés à A_T ?

HISTORIQUE

- Boruvka (1926) $\Rightarrow$ Kruskal (1956)
plusieurs parties de l'arbre couvrant de poids minimum sont construites à part,
- Jarnik (1930) $\Rightarrow$ Prim (1957)
à partir d'un seul sommet, l'arbre de recouvrement de poids minimum est construit.
- Algorithmes gloutons avec maintien d'un invariant (à chaque instant, le sous ensemble d'arêtes choisies est une partie de l'arbre couvrant de poids minimal), à chaque étape, une nouvelle "meilleure" arête est choisie.

PRINCIPE DE L'ALGORITHME DE KRUSKAL (1956)

Principe: construction concurrente d'arbres de poids minimum.

- Au départ, chaque sommet est un arbre,
- Les arêtes du graphe sont triées par ordre de poids croissant,
- Tant qu'il reste au moins deux arbres :
 - choix de la prochaine arête,
 - si l'arête joint deux arbres, ils sont fusionnés,

ALGORITHME DE KRUSKAL (1956)

1. Trier « dans un tableau E » et (ré)numéroter les arêtes de G par ordre croissant sur les coûts (départager arbitrairement en cas d'égalité).

$k = 1, l = 0$ et $A_T = \emptyset$ « ensemble des arêtes de l'A.C.M »

2. Si e_k ne forme pas de cycle avec les arêtes de A_T Faire

$$A_T = A_T \cup \{e_k\}$$

$$l = l + 1$$

3. Si $l = n - 1$ – FIN–

Sinon poser $k = k + 1$, et retour à 2

Complexité:

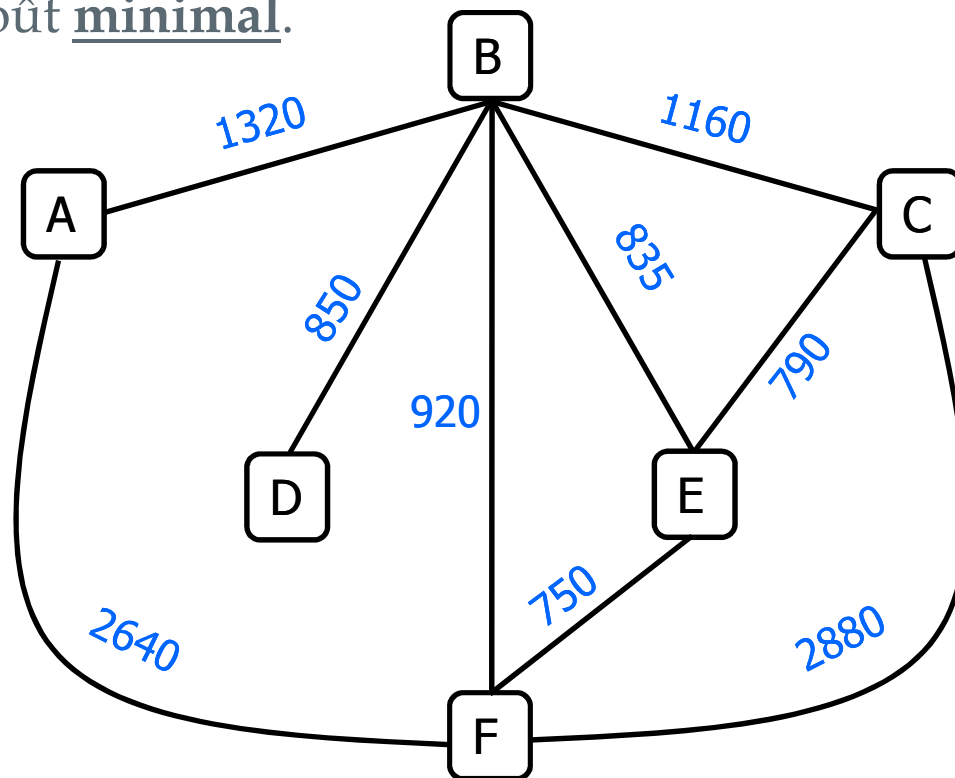
- Soit $G = (S; A)$ le graphe d'origine, avec $|S| = n$, et $|A| = m$, soit $T = (S; A_T)$ l'arbre couvrant
- la première étape consiste à trier l'ensemble des arêtes du graphe $\Rightarrow O(m \log m)$
- à chaque étape une nouvelle arête est considérée, il y a donc exactement m étapes
- l'arête de plus faible poids est choisie elle est insérée après vérification qui nécessite $O(1)$,



Complexité totale en $O(m \log(m))$

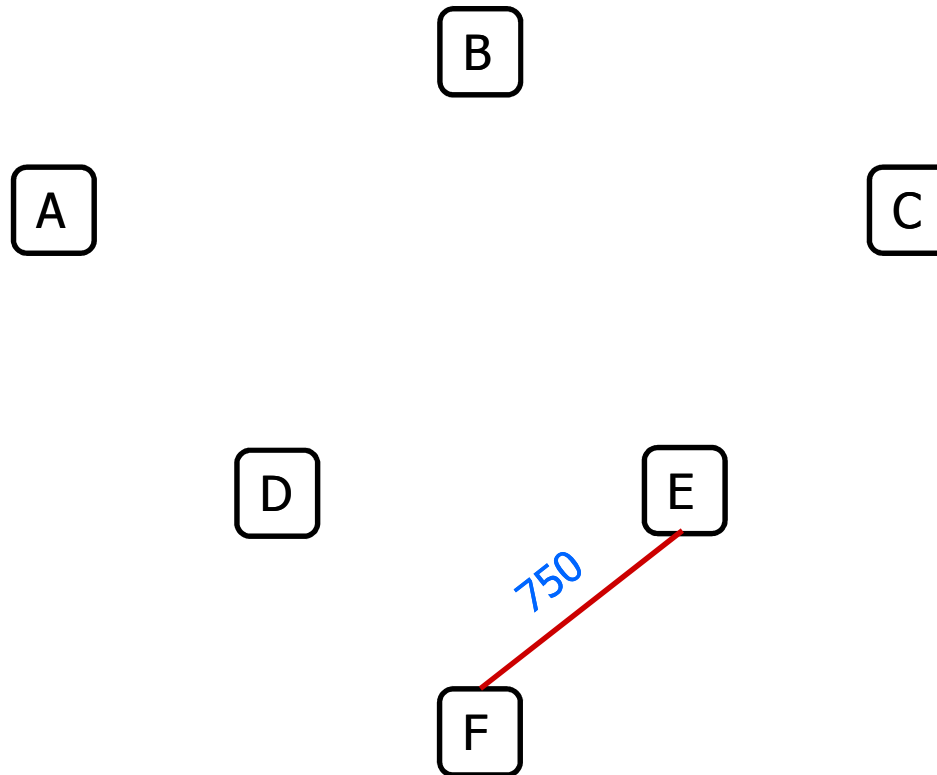
EXEMPLE

Situation qui illustre les coûts estimés de la construction de trottoirs entre des immeubles. On veut s'assurer que tous les immeubles sont reliés à un coût minimal.



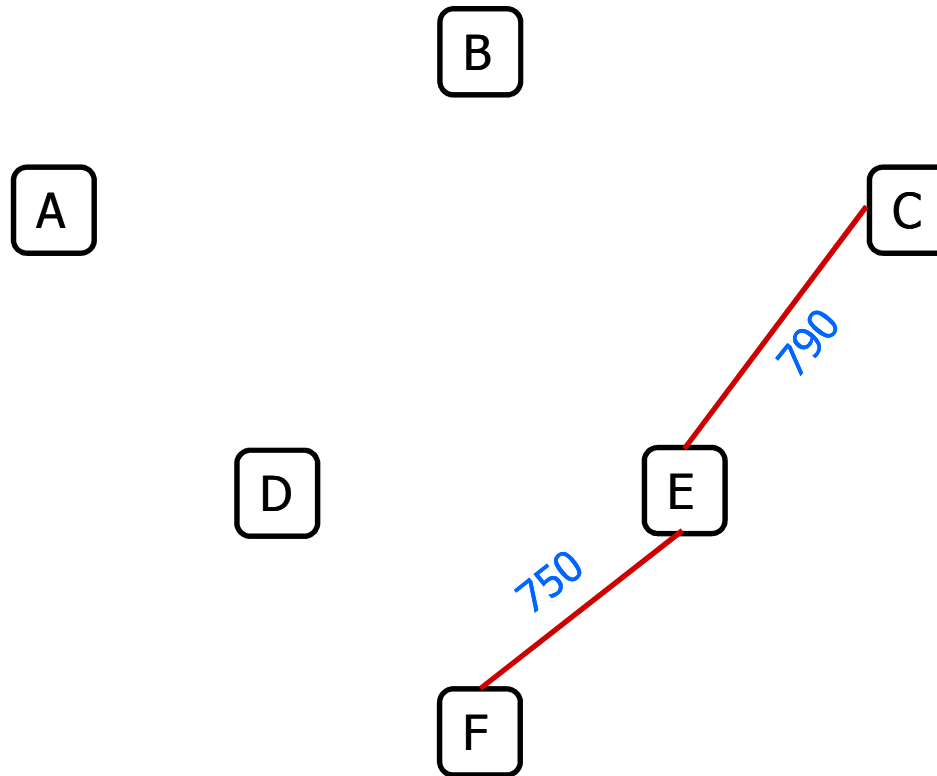
MÉTHODE : Algorithme de Kruskal

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



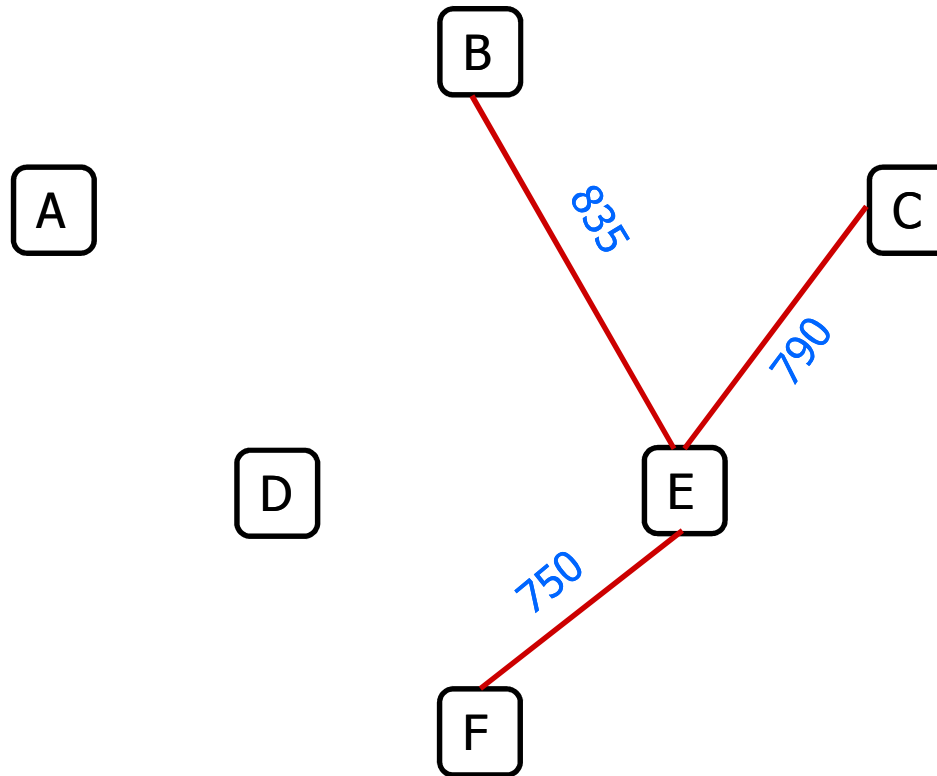
2. Arête avec le plus petit poids :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



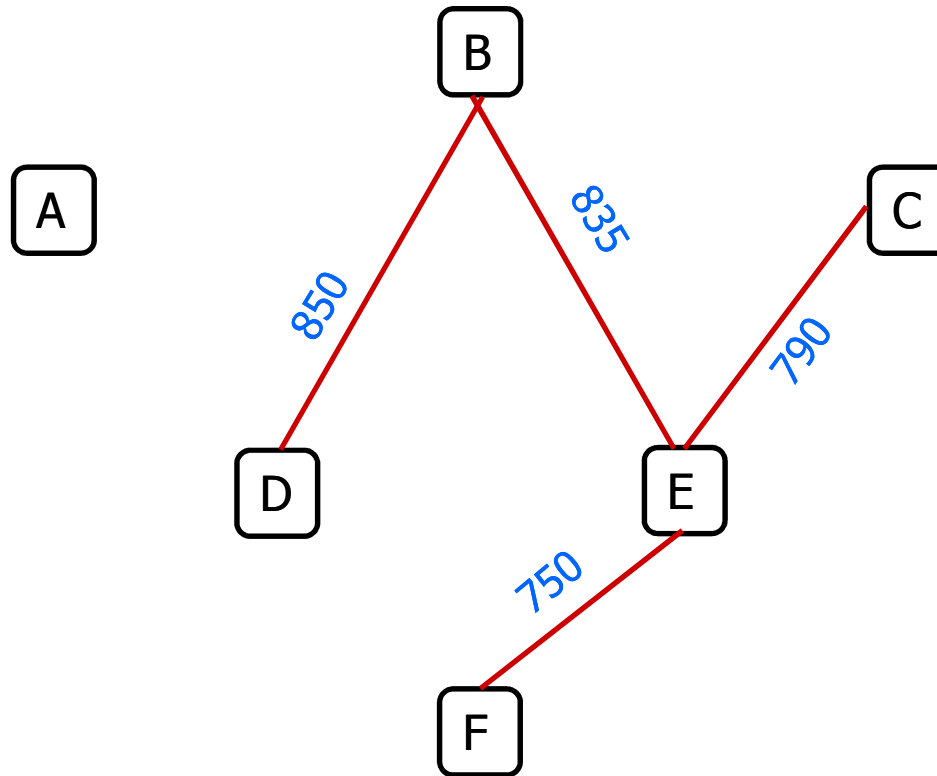
3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



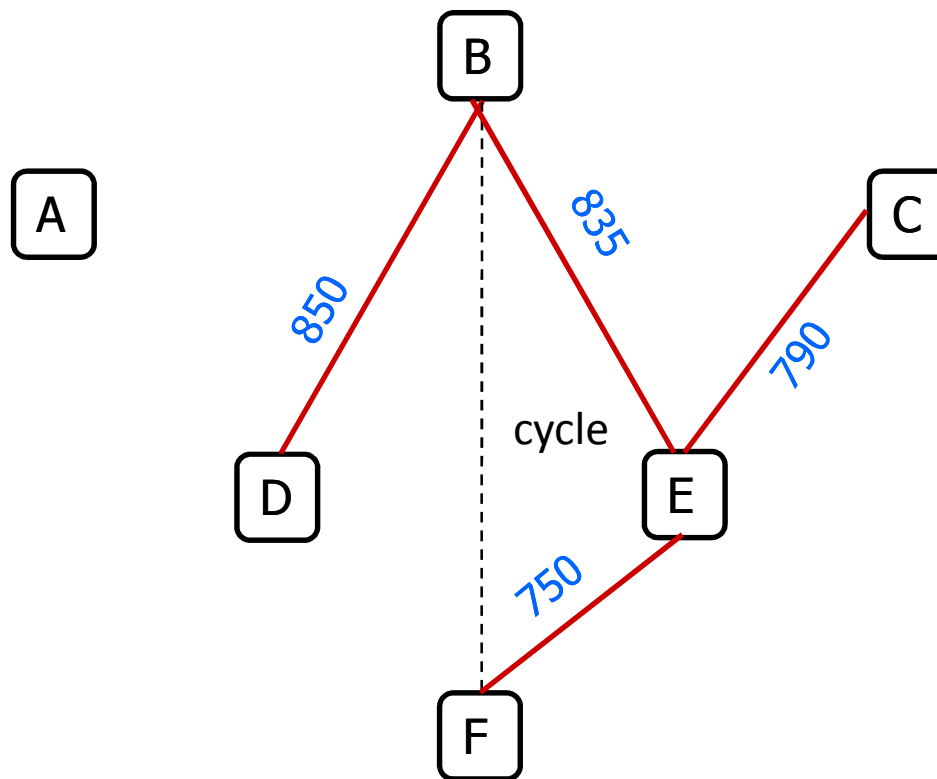
3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



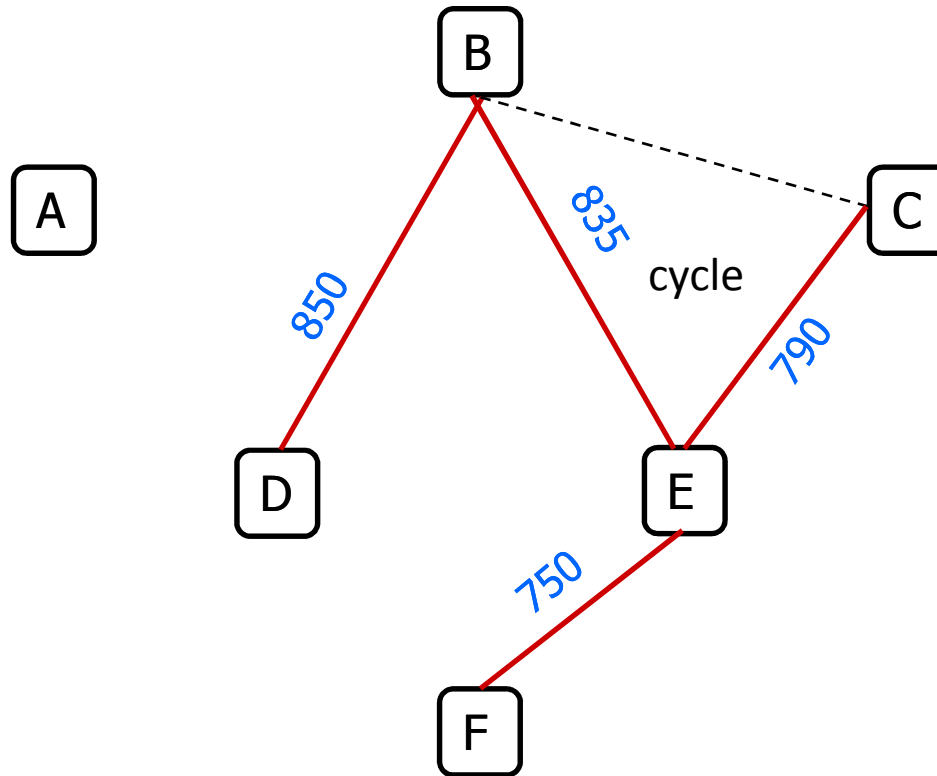
3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



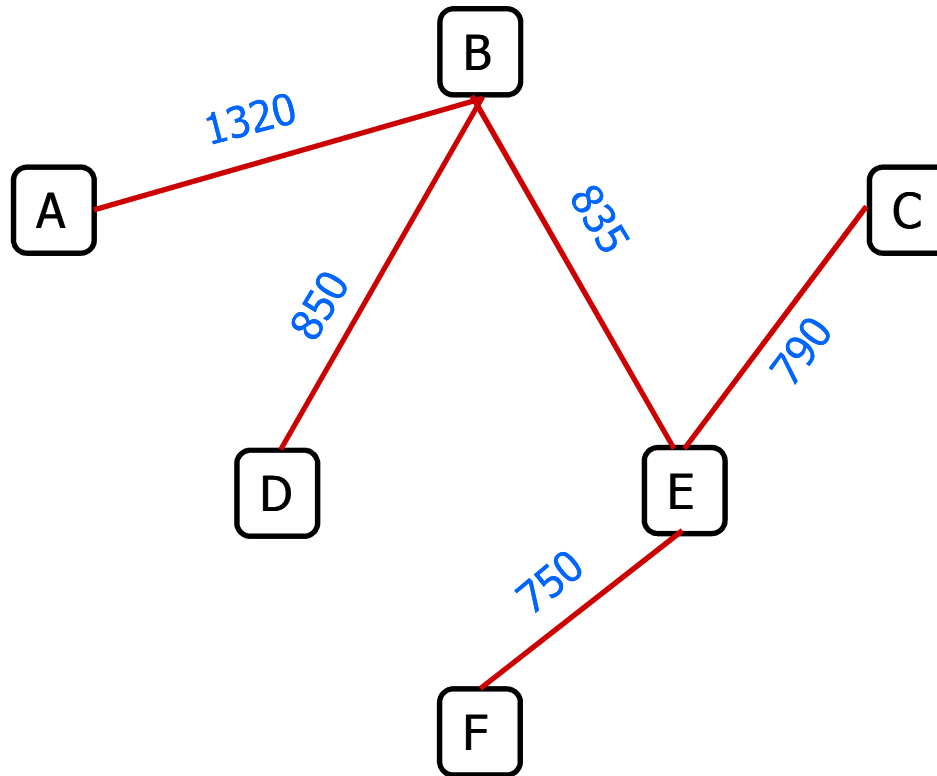
3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



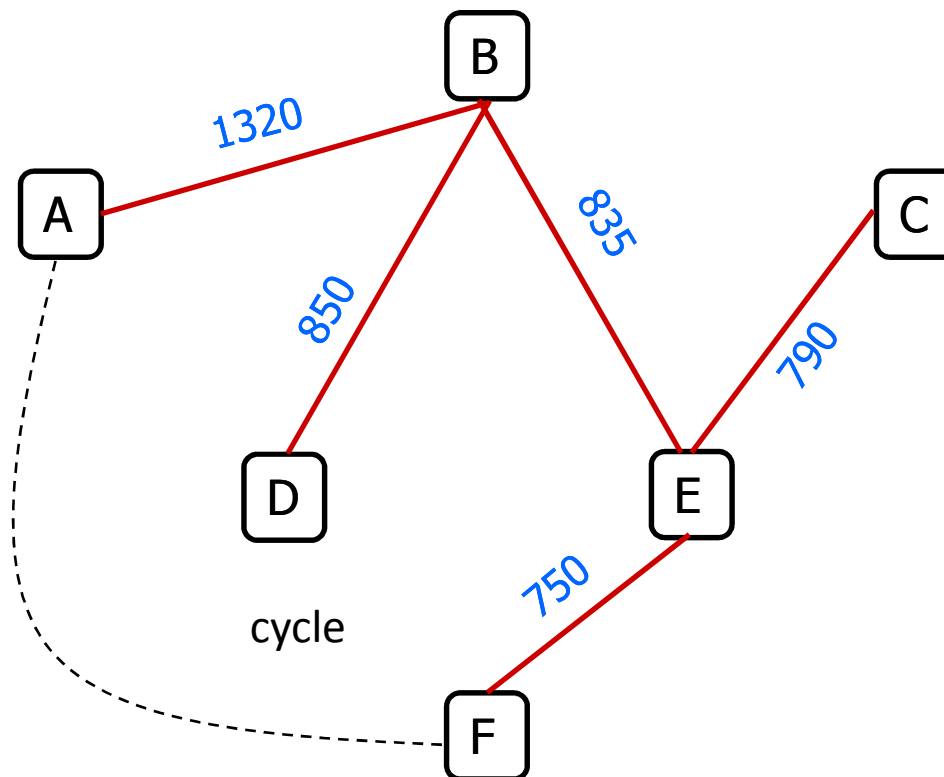
3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



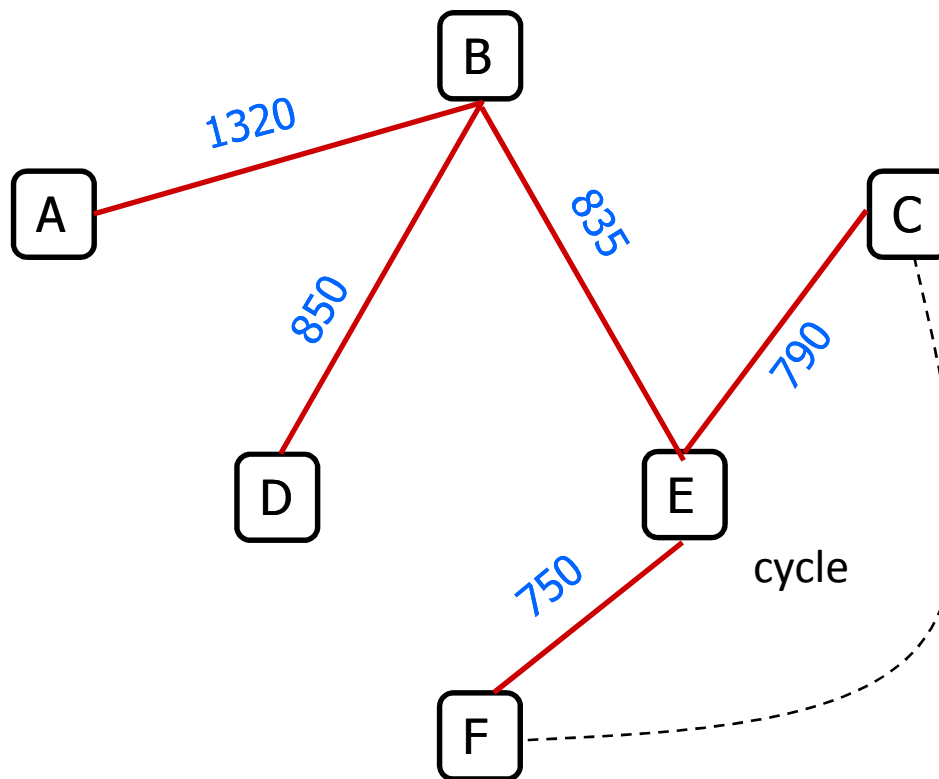
3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880



3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880

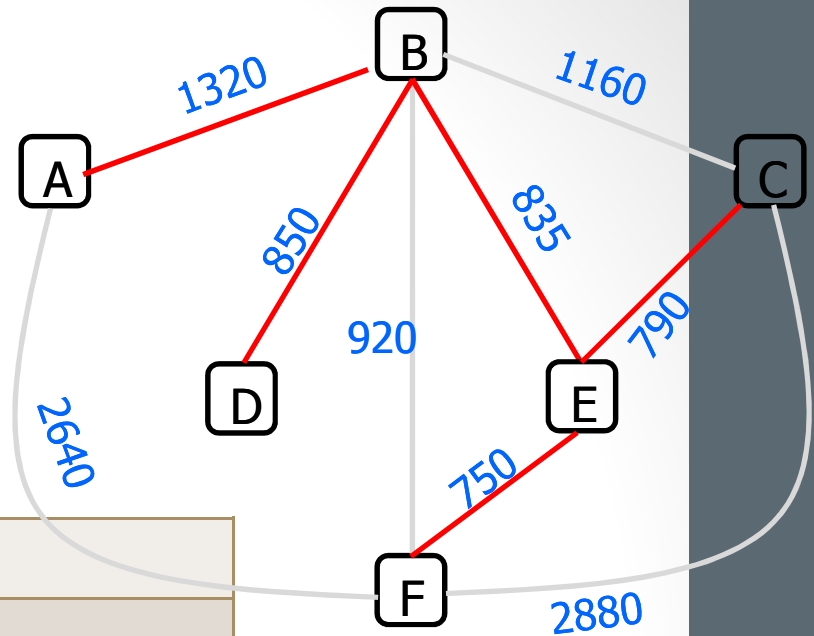


3. Répéter en évitant de former un cycle simple :

EF	CE	BE	BD	BF	BC	BA	AF	CF
750	790	835	850	920	1160	1320	2640	2880

Déroulement de l'algorithme de Kruskal:

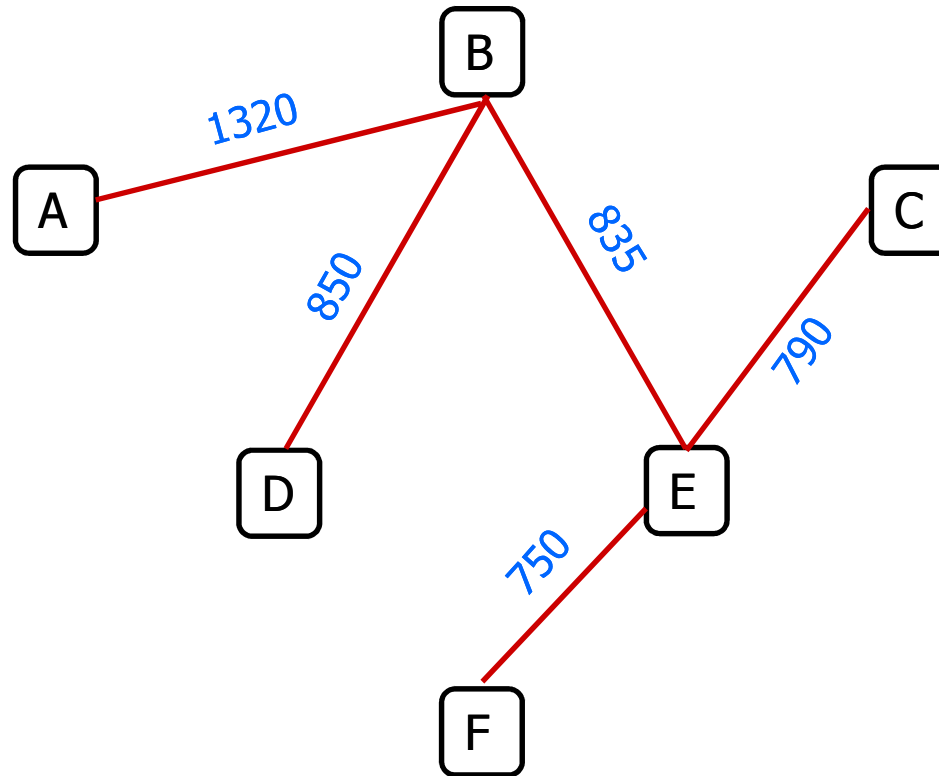
k	1	2	3	4	5	6	7	8	9
e_k	EF	CE	BE	BD	BF	BC	BA	AF	CF



A_T	I	k	e_k
$\emptyset$	0	1	EF
$A_T \cup \{EF\}$	1	2	CE ne forme pas de cycle ✓
$A_T \cup \{CE\}$	2	3	BE ne forme pas de cycle ✓
$A_T \cup \{BE\}$	3	4	BD ne forme pas de cycle ✓
$A_T \cup \{BD\}$	4	5	BF forme un cycle ✗
		6	BC forme un cycle ✗
		7	BA ne forme pas de cycle ✓
$A_T \cup \{BA\}$	$5=n-1$	FIN	

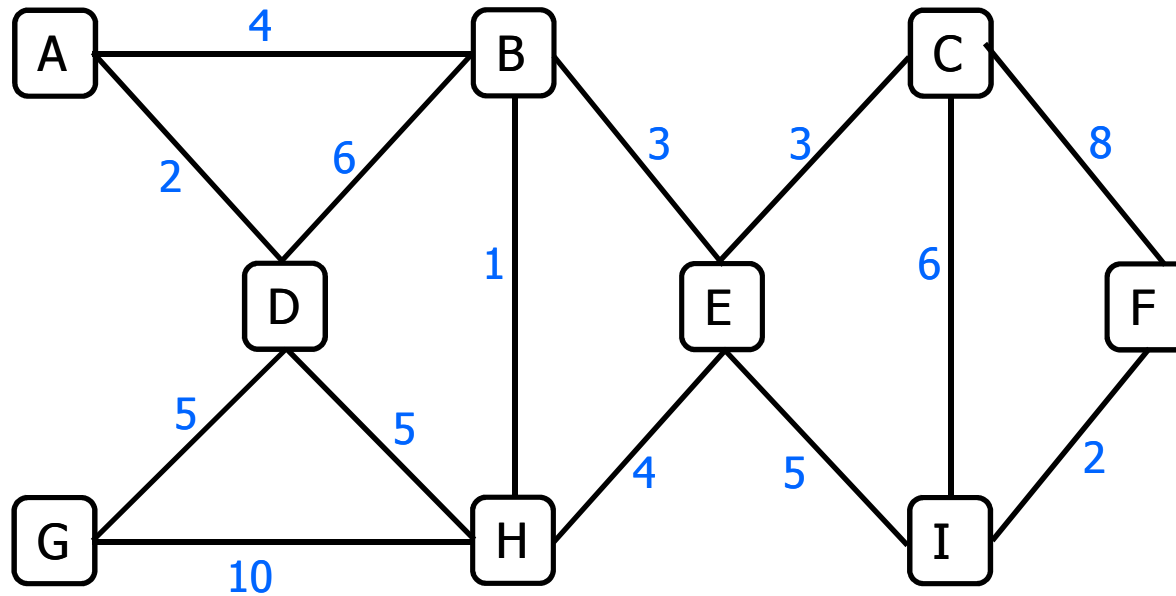
La
construction
des trottoirs
coûtera donc
4545 ...

L'arbre couvrant T^* de poids minimal est :



Poids de l'arbre : $750 + 790 + 835 + 850 + 1320 = 4545$

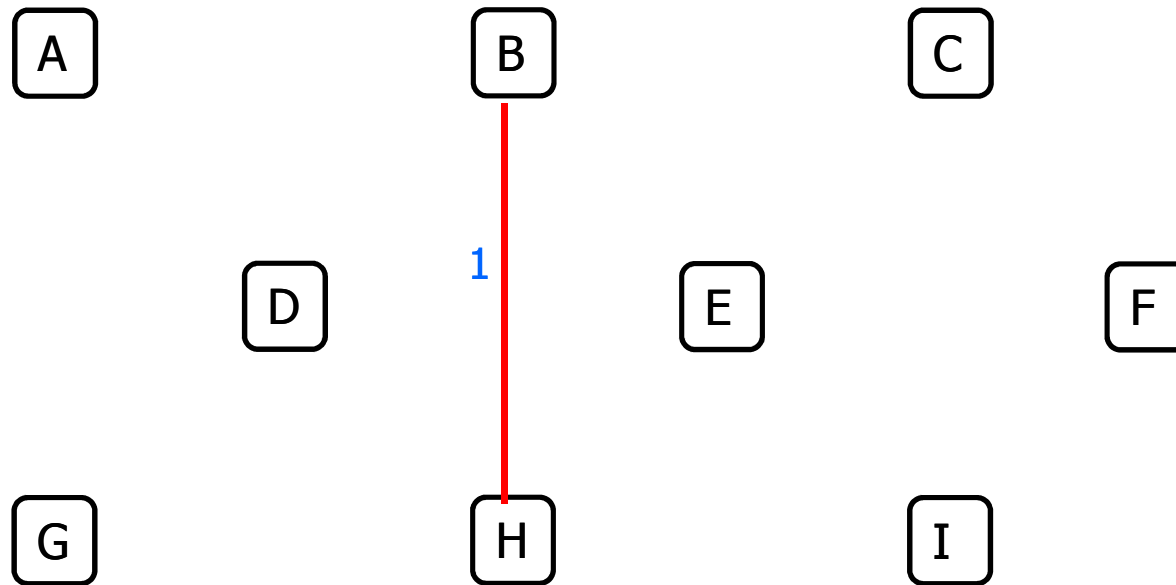
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4	5	6	8	10						

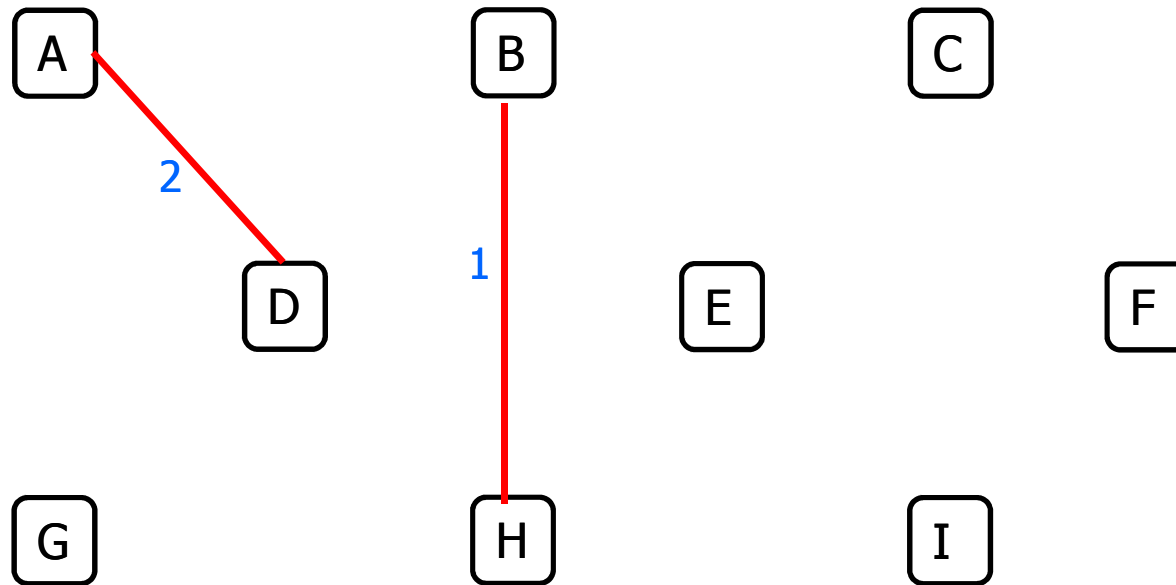
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4	5	6	8	10						

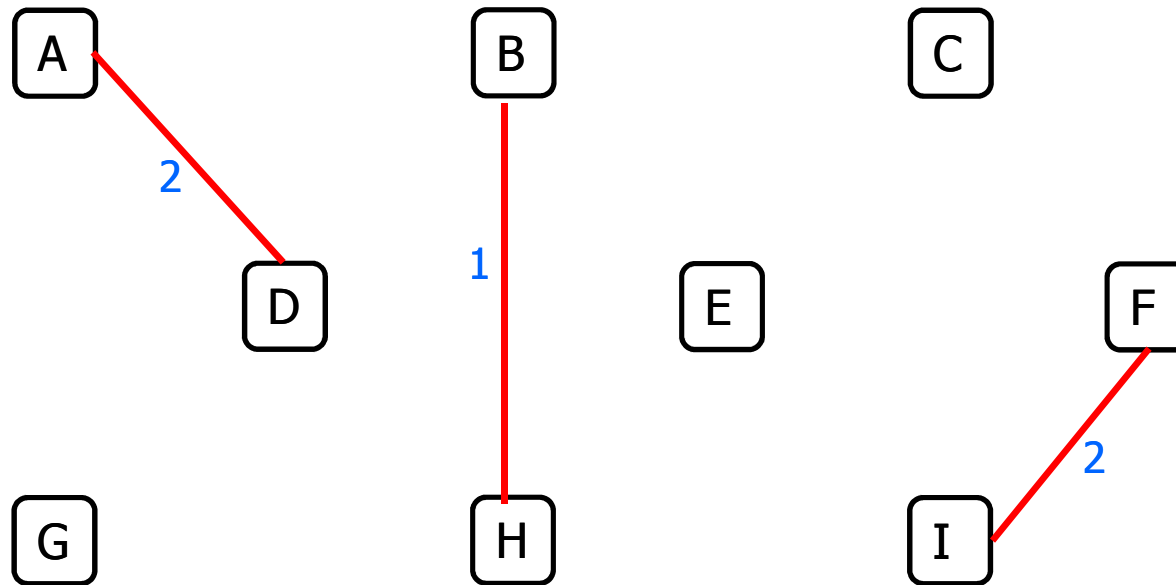
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4	5	6	8	10						

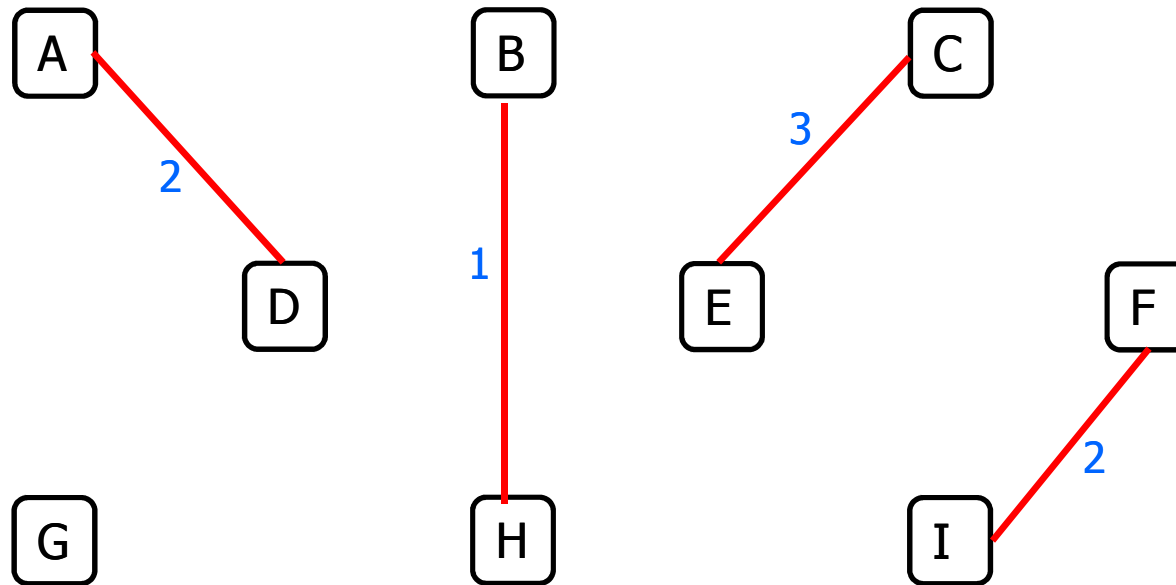
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4	5	6	8	10						

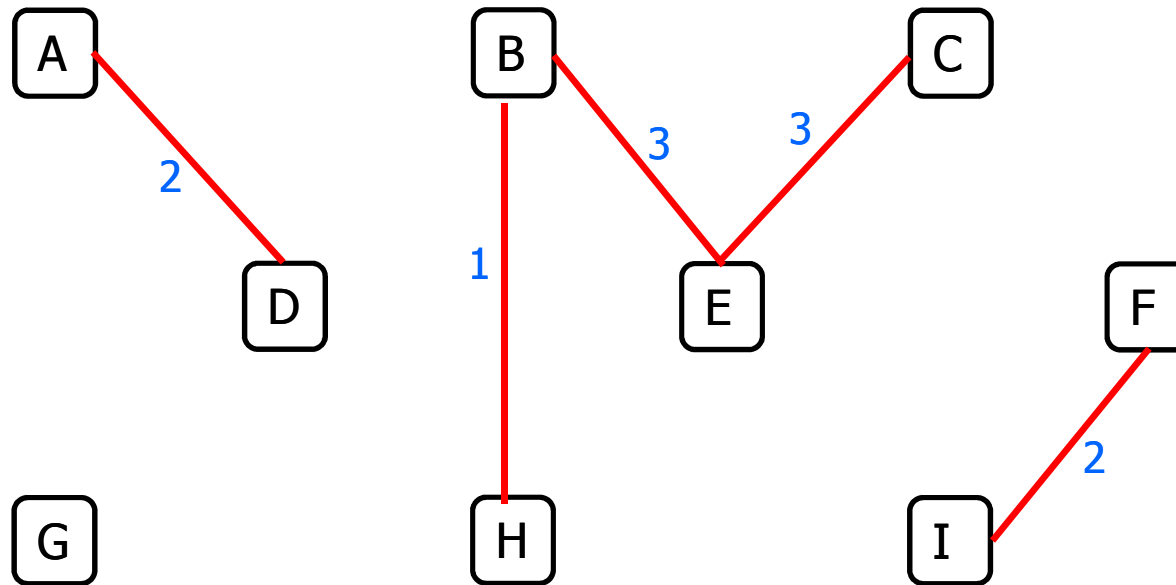
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3		4		5		6	8	10			

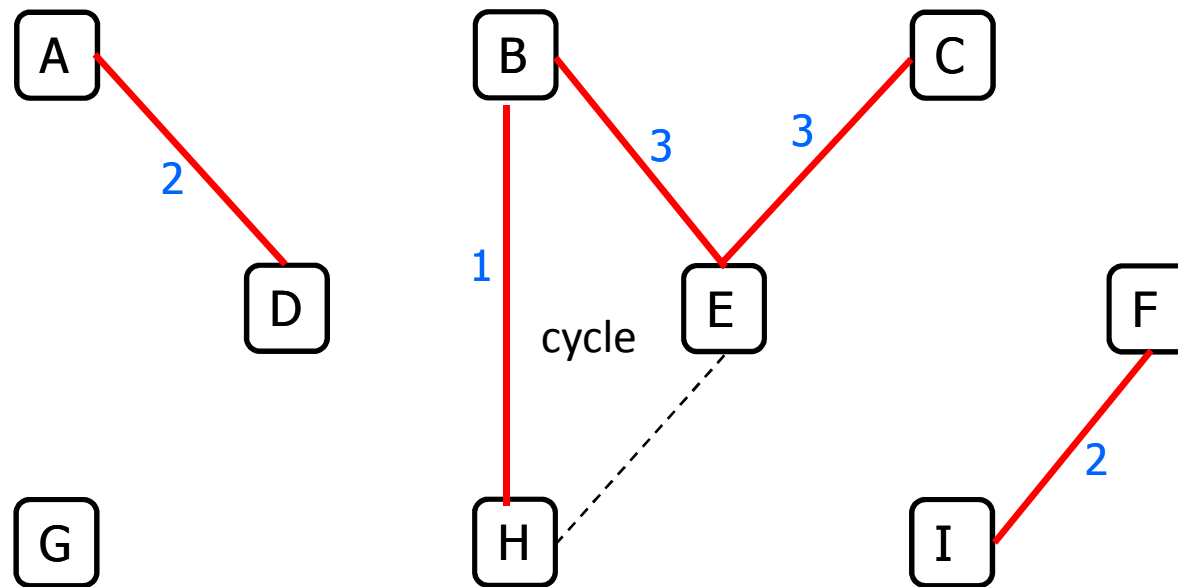
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4	5	6	8	10						

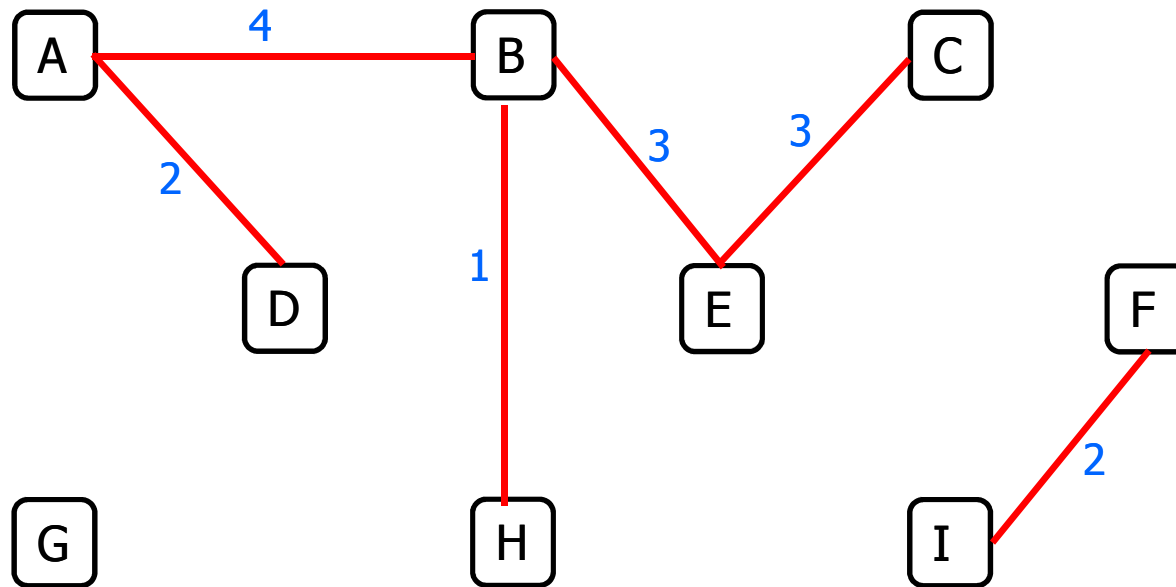
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4					5		6		8	10

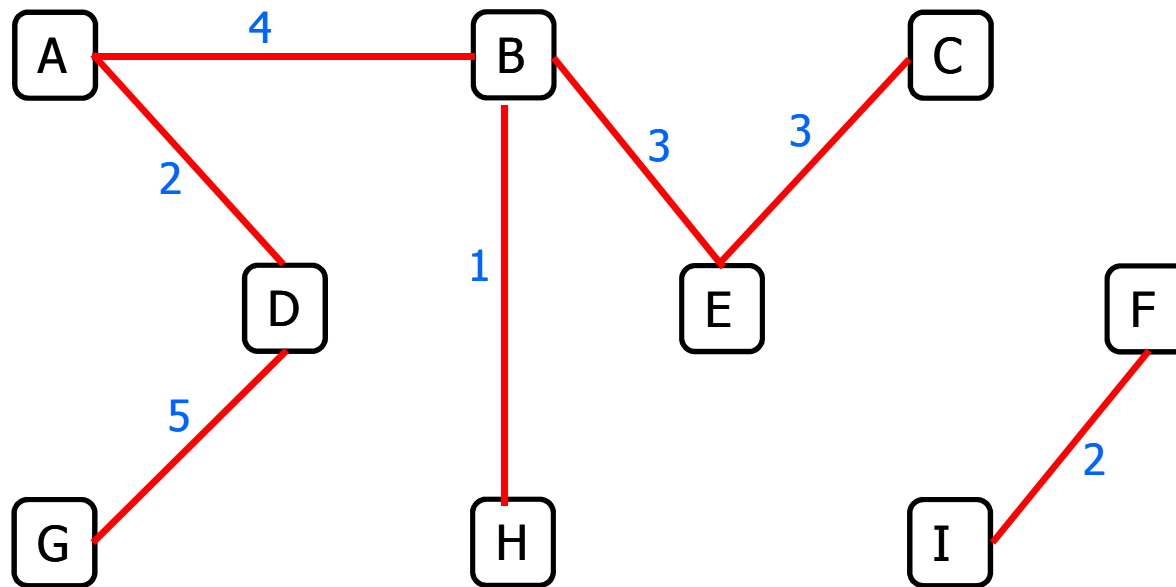
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4				5			6	8	10	

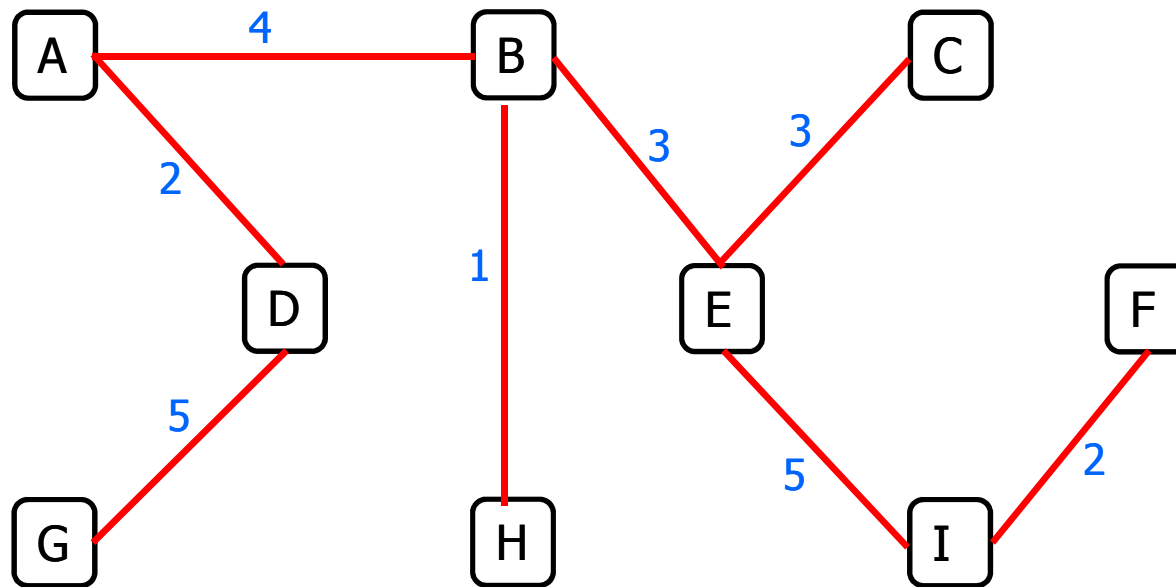
Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Répéter en évitant de former un cycle simple :

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4			5		6	8	10			

Exercice #1 : Détermine l'arbre de valeur minimale ainsi que son poids.



Inutiles car on a déjà un graphe connexe avec $8(=n-1)$ arrêtes

BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
1	2	3	4	5		6	8	10					

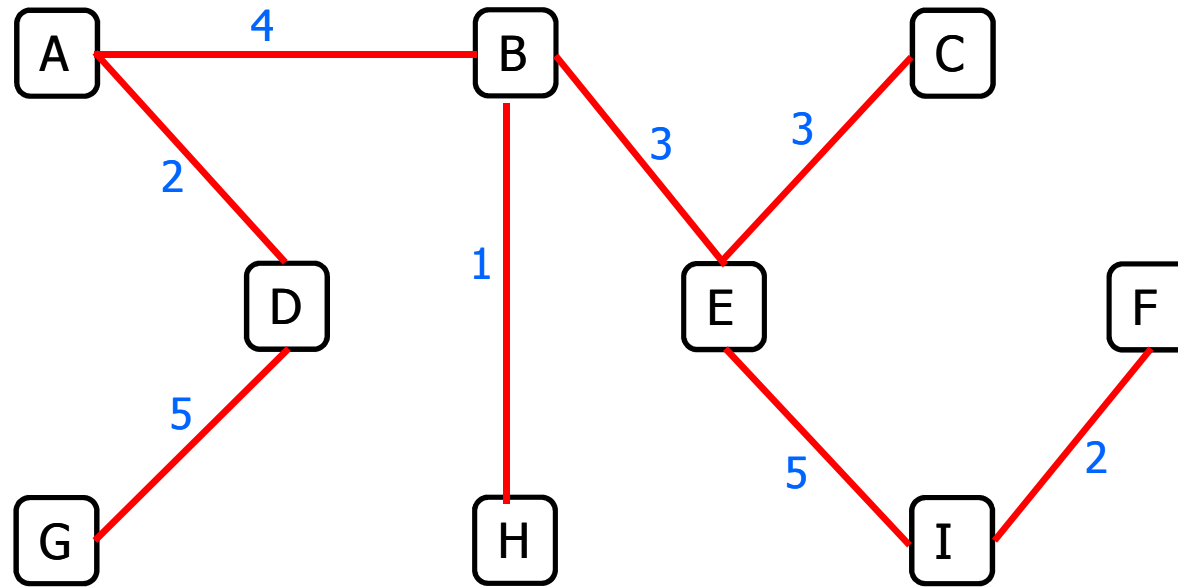
Algo Kruskal converge en $n-1$ itérations $\Rightarrow$ FIN

Déroulement de l'algorithme de Kruskal:

e_k	BH	AD	FI	CE	BE	HE	AB	DG	EI	DH	BD	CI	CF	GH
k	1	2	3	4	5	6	7	8	9	10	11	12	13	14

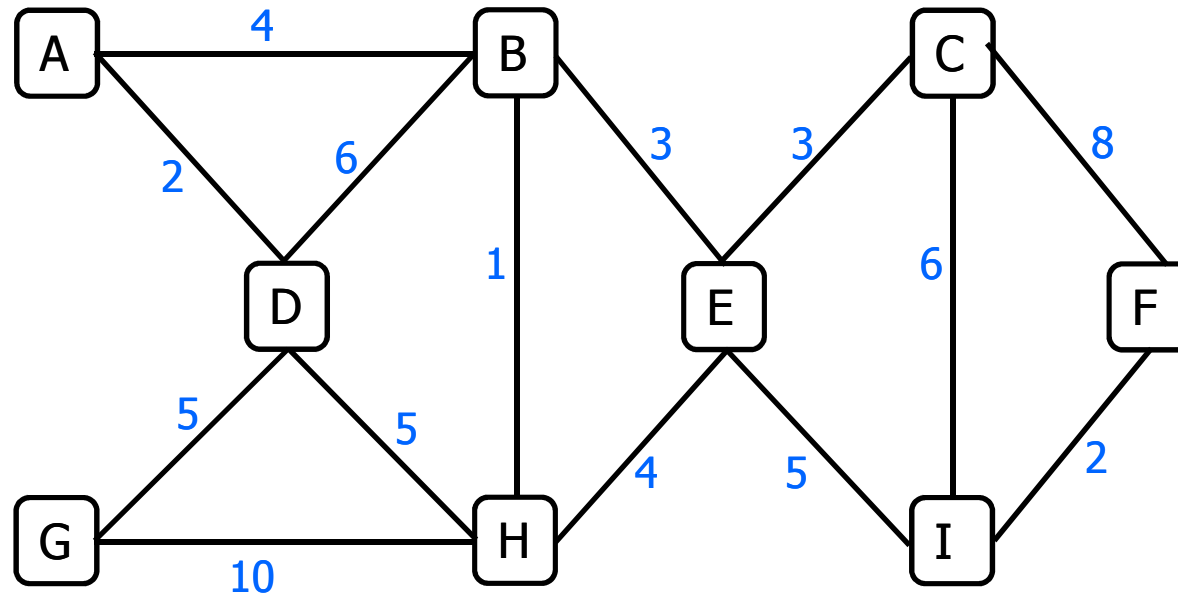
A_T	l	k	e_k
$\emptyset$	0	1	BH
$A_T \cup \{BH\}$	1	2	AD ne forme pas de cycle ✓
$A_T \cup \{AD\}$	2	3	FI ne forme pas de cycle ✓
$A_T \cup \{FI\}$	3	4	CE ne forme pas de cycle ✓
$A_T \cup \{CE\}$	4	5	BE ne forme pas de cycle ✓
$A_T \cup \{BE\}$	5	6	HE forme un cycle ✗
		7	AB ne forme pas de cycle ✓
$A_T \cup \{AB\}$	6	8	DG ne forme pas de cycle ✓
$A_T \cup \{DG\}$	7	9	EI ne forme pas de cycle ✓
$A_T \cup \{EI\}$	8=n-1	FIN	

Exercice #1 : L'arbre couvrant T de poids minimal.

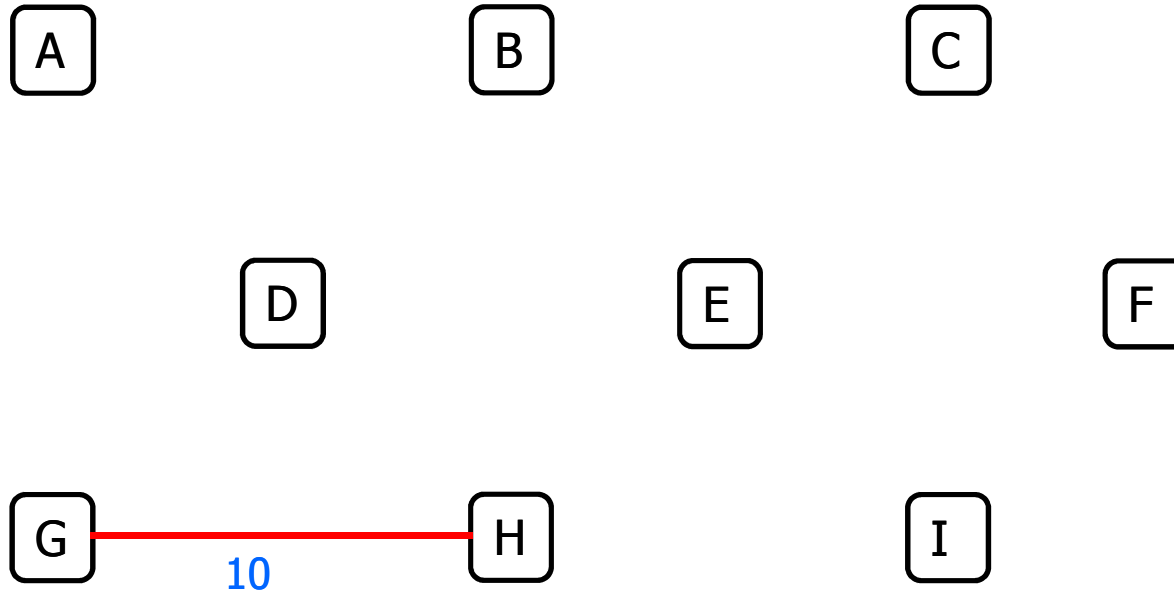


Poids de l'arbre : $1 + 2 + 2 + 3 + 3 + 4 + 5 + 5 = 25$

Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



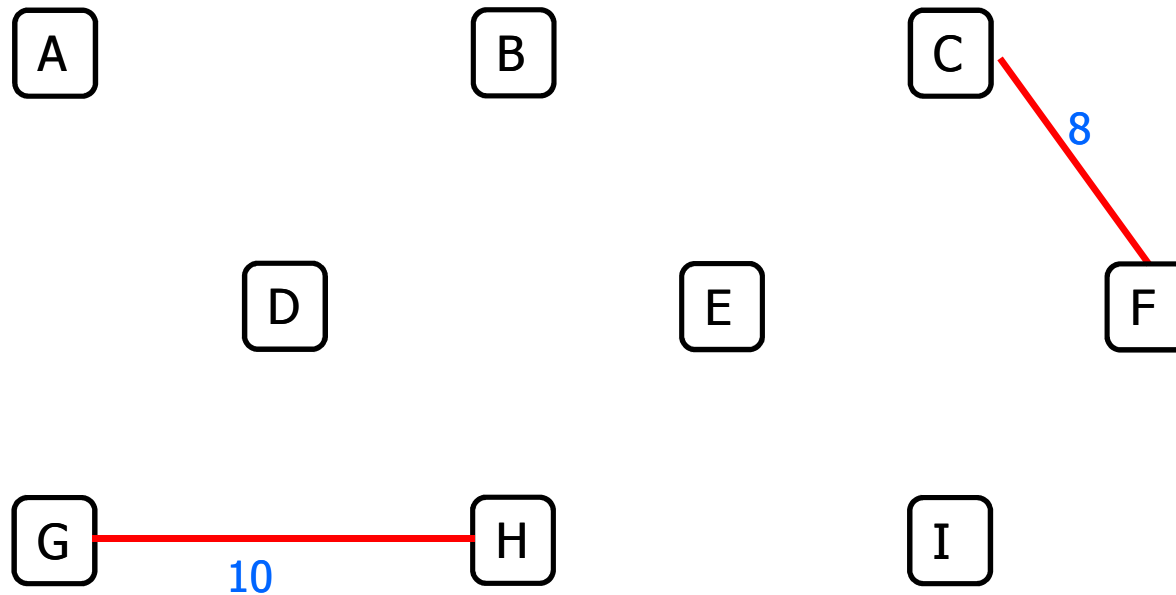
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6	5			4		3		2		1	

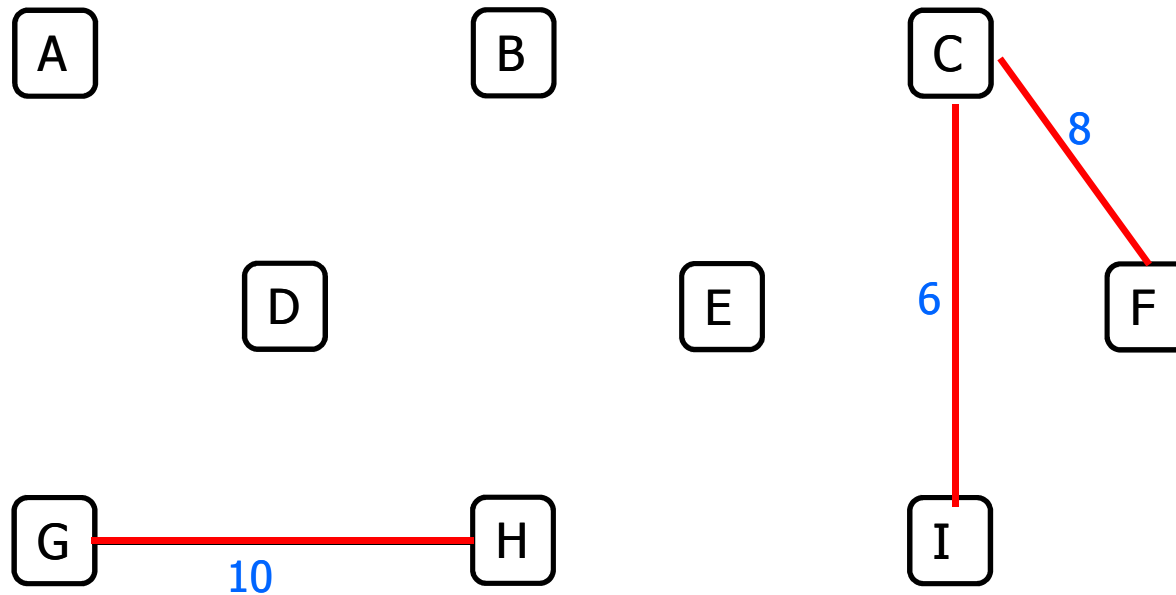
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6	5	4	3	2	1						

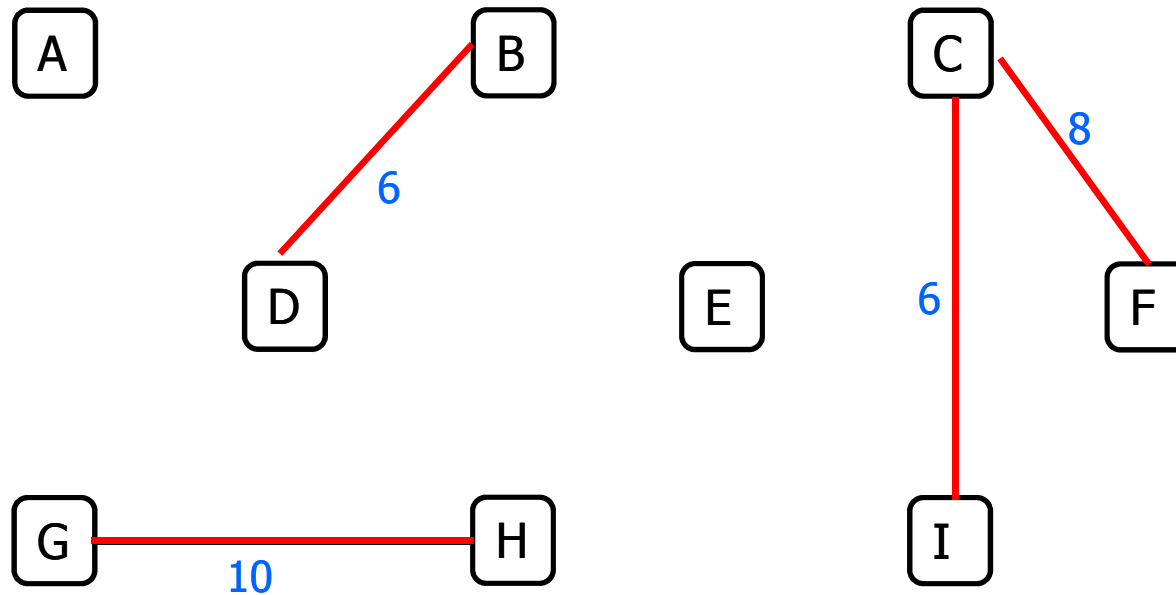
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6	5			4		3		2		1	

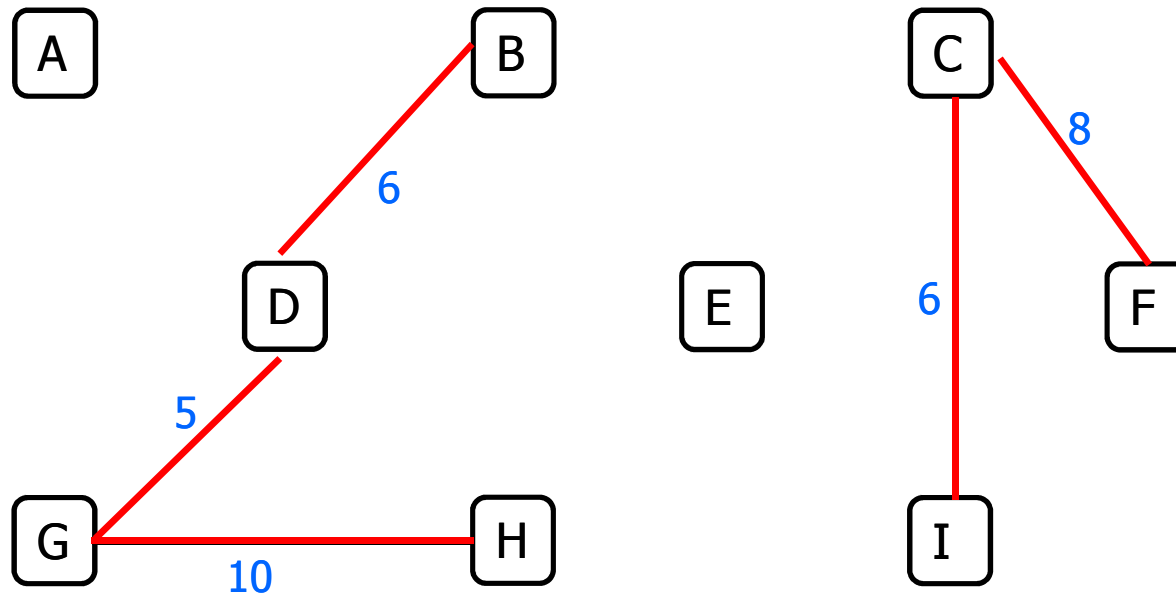
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6		5			4		3		2		1

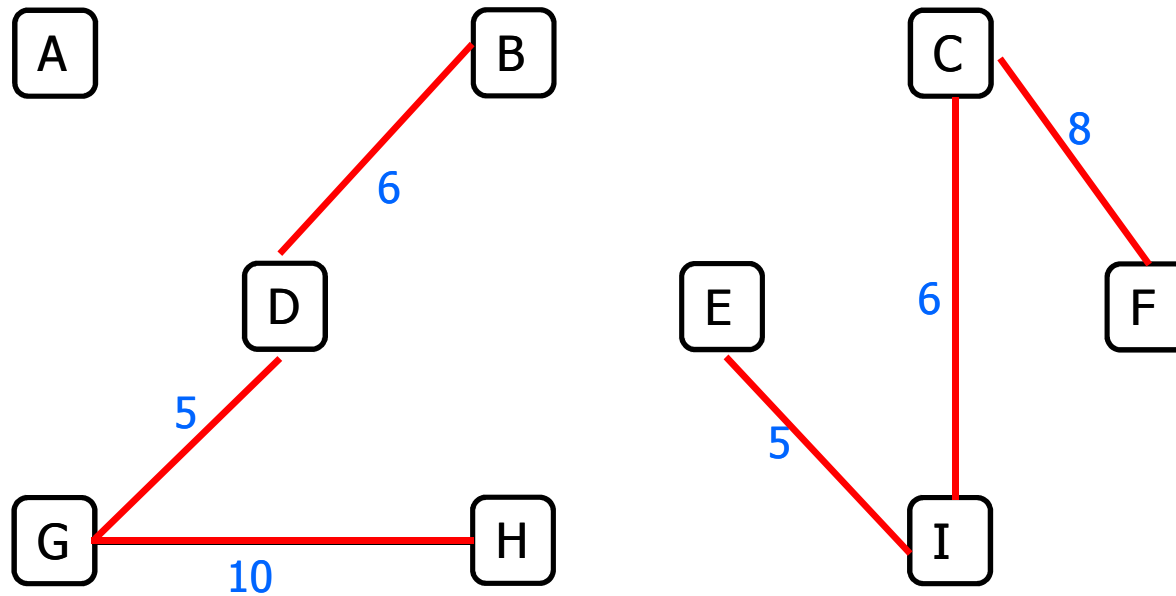
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6	5			4		3		2		1	

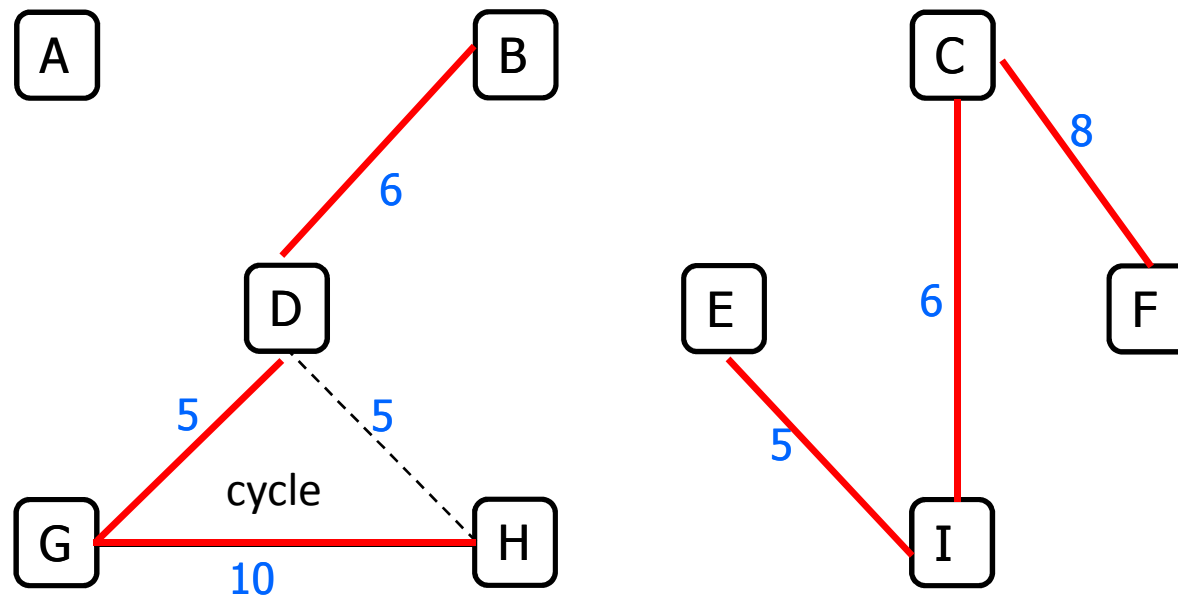
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6	5	5	5	4	4	3	3	2	2	1	1

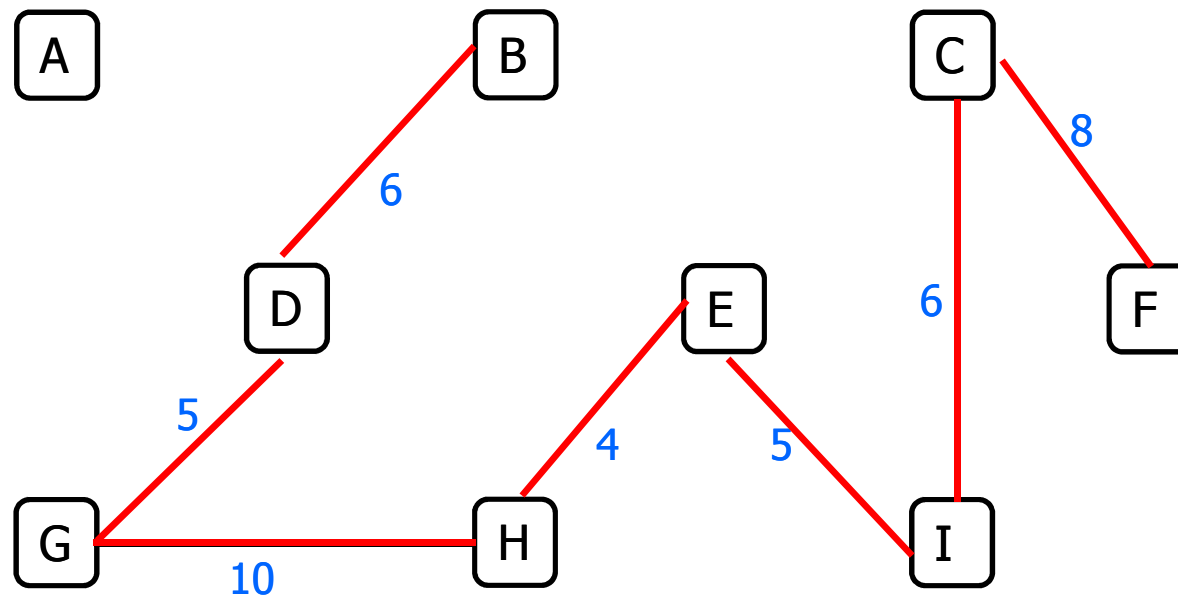
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6	5	5	5		4		3		2		1

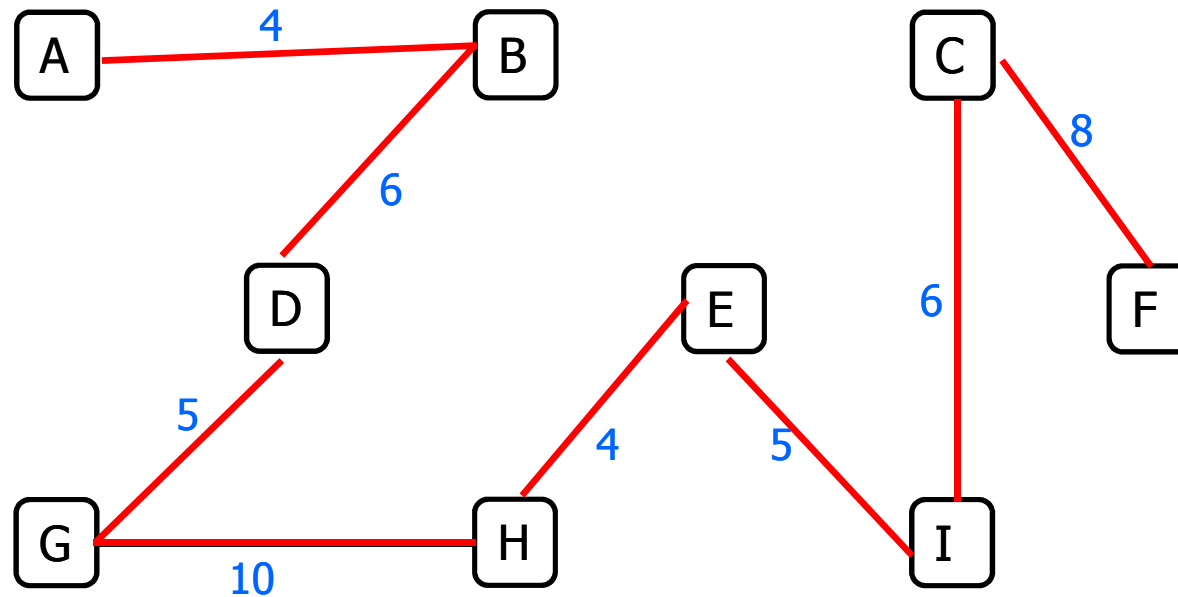
Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.



MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6	5				4		3		2		1

Exercice #2 : Détermine l'arbre de valeur maximale ainsi que son poids.

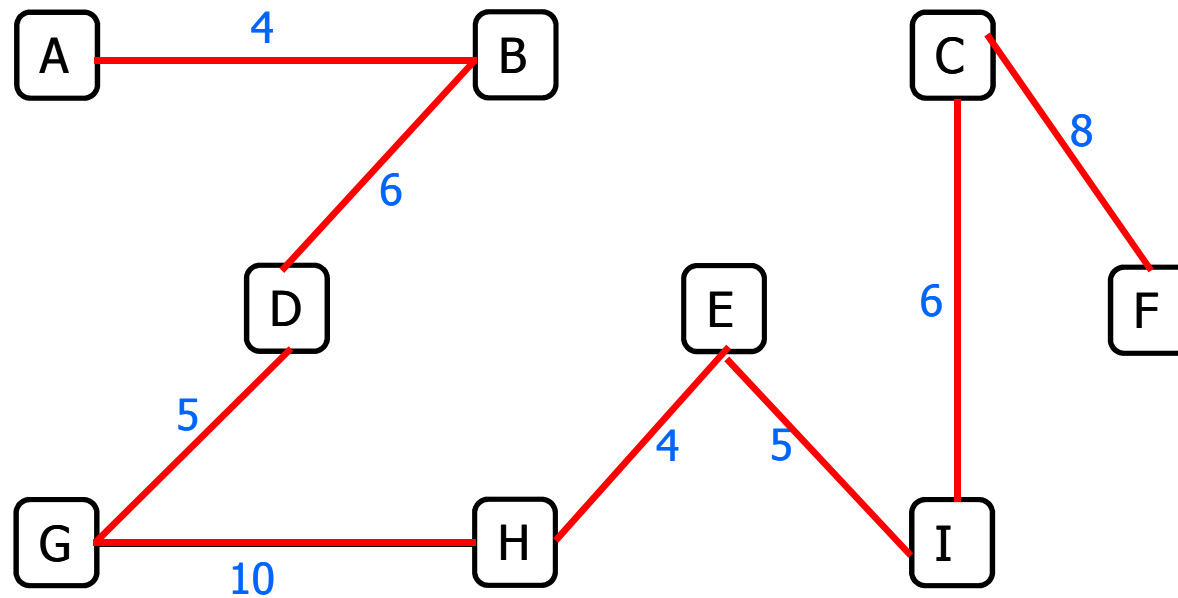


MÉTHODE : Algorithme de Kruskal (arrêtes par ordre décroissant)

GH	CF	CI	BD	DG	EI	DH	HE	AB	CE	BE	AD	FI	BH
10	8	6		5			4		3		2		1

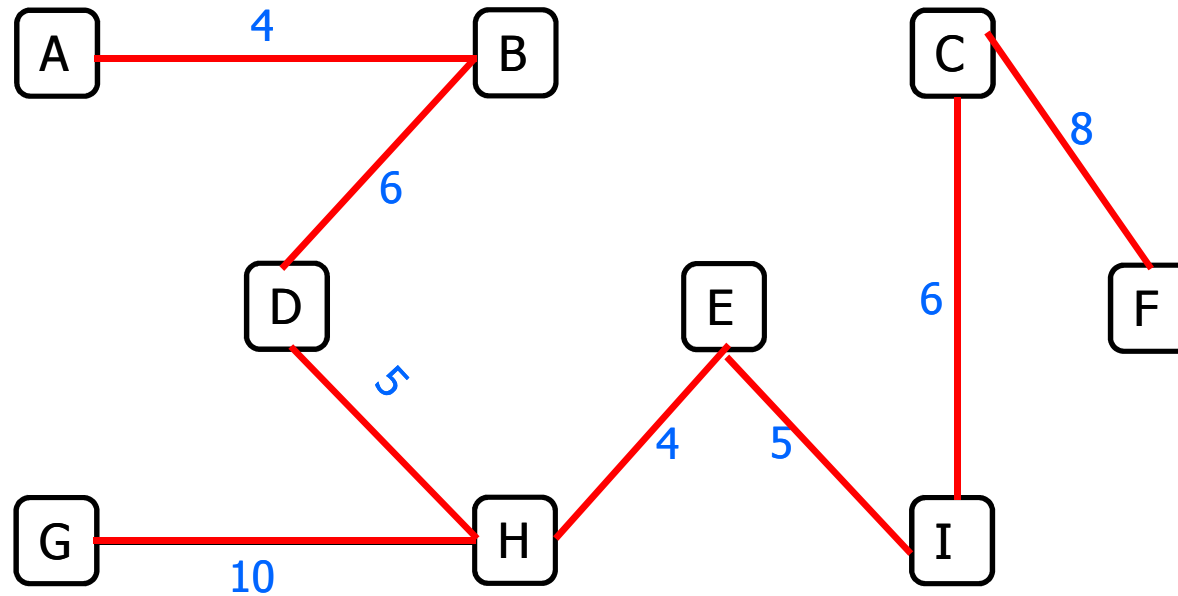
inutiles

Exercice #2 : L'arbre couvrant de poids maximal :



Poids de l'arbre : $10 + 8 + 6 + 6 + 5 + 5 + 4 + 4 = 48$

Remarque : pour cet exemple, l'arbre couvrant de poids maximal, n'est pas unique. En effet



Poids de l'arbre : $10 + 8 + 6 + 6 + 5 + 5 + 4 + 4 = 48$

Principe de l'Algorithme de Prim (1957)

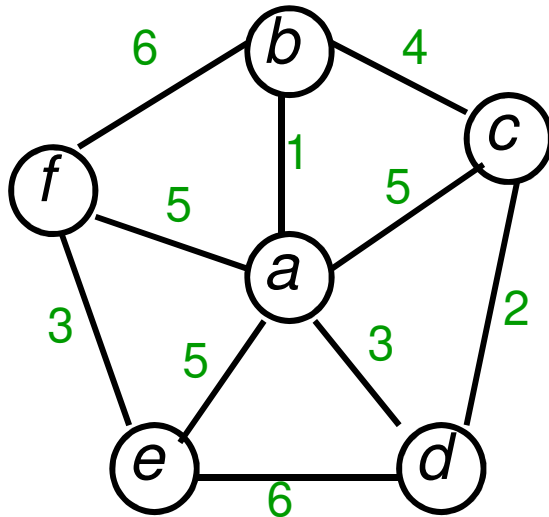
- construction incrémentale d'un arbre de poids minimum
- au départ, un sommet s_i est choisi arbitrairement, ce sommet constitue l'arbre couvrant de poids minimum
- parmi toutes les arêtes incidentes à s_i , choisir celle de plus faible poids $(s_i ; s_j)$

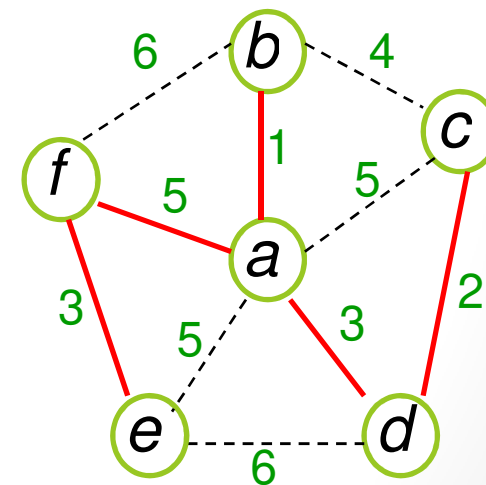
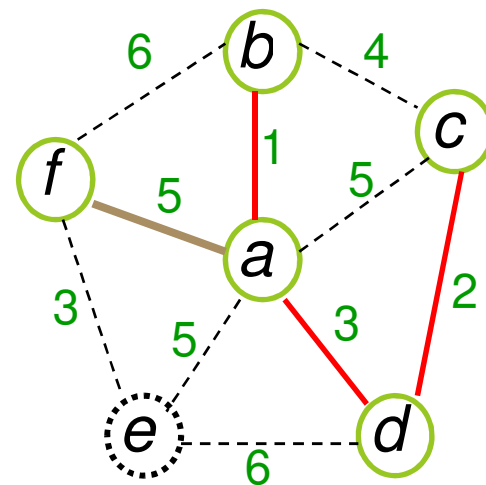
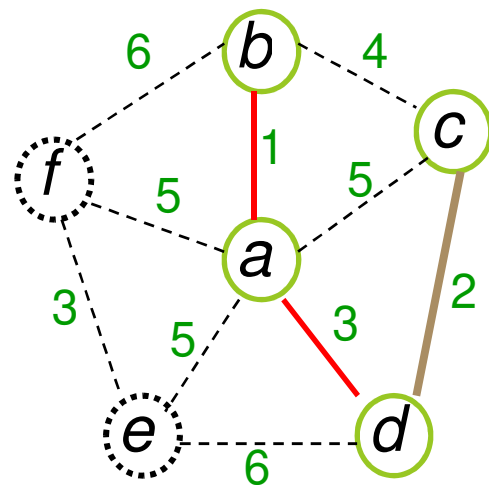
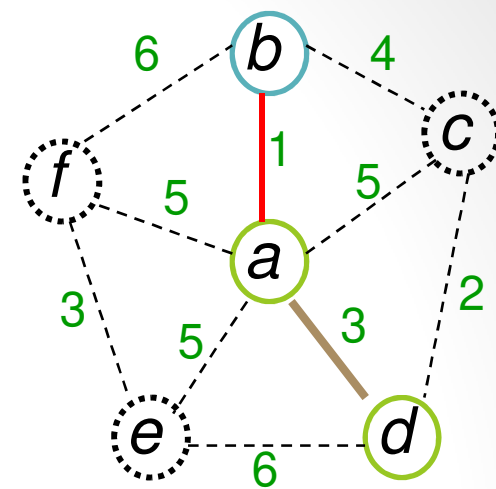
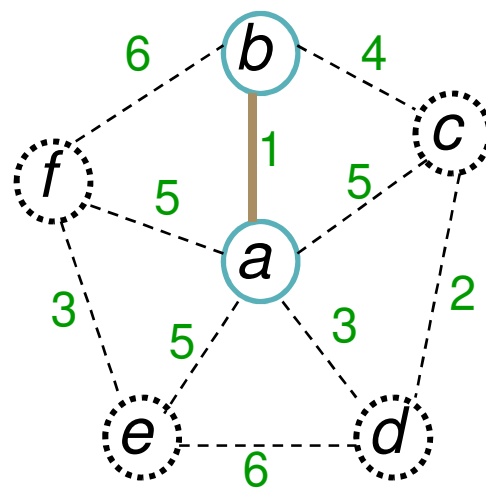
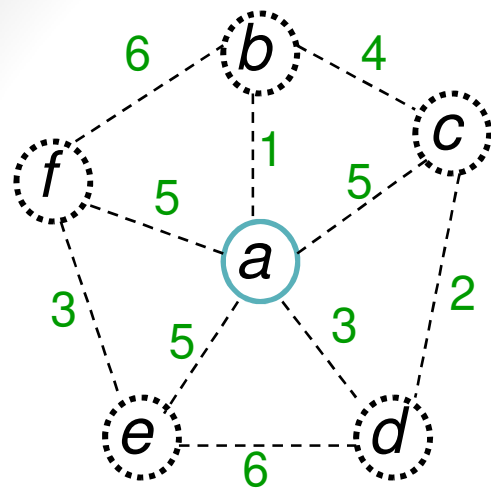
 le nouvel arbre obtenu est constitué des sommets s_i et s_j et de l'arête $(s_i ; s_j)$

- tant qu'il reste des sommets en dehors de l'arbre :
parmi l'ensemble des arêtes incidentes aux sommets de l'arbre et ayant une extrémité en dehors de l'arbre, choisir celle dont le poids est le plus faible

Exemple

Principe de l'algo de Prim : faire grossir un sous-arbre jusqu'au recouvrement du graphe





ALGORITHME DE PRIM

1- Initialisation:

$$S_T = \{s\},$$

« s : est un sommet quelconque de S »

$$k := 0, \quad \pi_k(s) = 0$$

$$\forall i \in \bar{S}_T \quad \pi_k(i) = \begin{cases} c_{si} & \text{si } i \text{ adjacent à } s \\ +\infty & \text{si non} \end{cases}$$

2- Sélectionner $j \in \bar{S}_T / \pi_k(j) = \min \{ \pi_k(i); i \in \bar{S}_T \}$

Faire $S_T := S_T \cup \{j\}$ si $S_T = S \Rightarrow \text{Fin.}$
sinon aller à 3

3- pour tout $i \in \bar{S}_T$ adjacent à j **Faire**

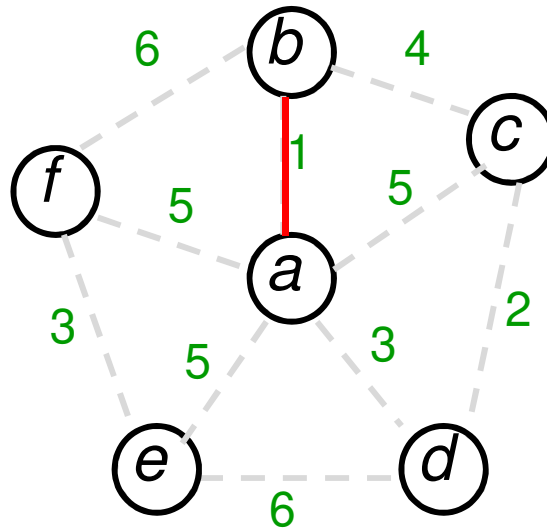
$$k := k + 1, \quad \pi_k(i) = \min \left(\pi_{k-1}(i), c_{ji} \right),$$

et Retour à 2

Complexité : de l'algorithme de Prim

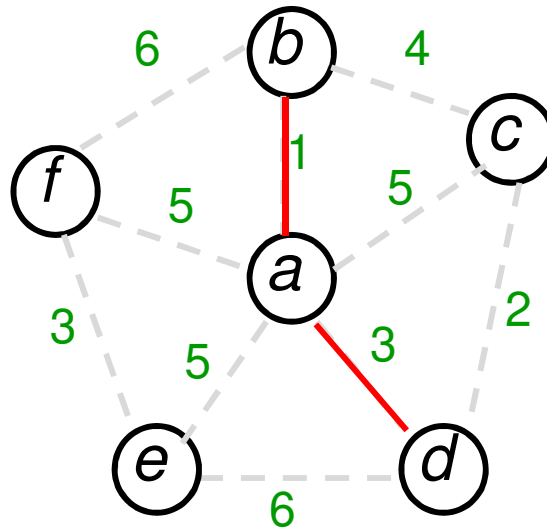
- La complexité est la même que pour l'algorithme de Dijkstra.
 - Si le graphe a n sommets et m arêtes, l'initialisation est en $O(n)$.
 - Il y a $n-1$ itérations. La recherche du minimum est en $O(n)$ chaque fois, soit $O(n^2)$ en tout.
 - Quand à l'actualisation, elle nécessite au total l'examen de toutes les arêtes dans les deux sens. Soit $2m$. l'actualisation d'une valeur est en temps constant d'où $O(m)$.
-
- Puisque typiquement $n < m < n^2$  Complexité totale en $O(n^2)$
 - Ceci est vrai si le graphe est représenté par un tableau des successeurs.

Retour à l'Exemple : avec sommet initial $s=b$



S_T	a	b	c	d	e	f
$\{b\}$	1	0	4	$+\infty$	$+\infty$	6

$$j \in \bar{S}_T / \pi_k(j) = \min \{ \pi_k(i); i \in \bar{S}_T \}$$

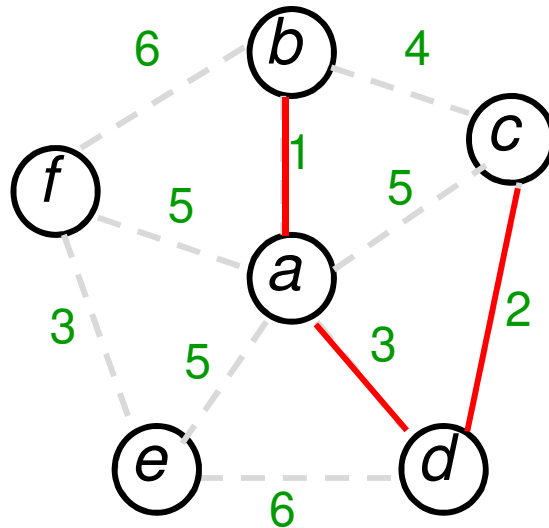


S_T	a	b	c	d	e	f
{b}	1	0	4	$+\infty$	$+\infty$	6
{b,a}			4	3	5	5

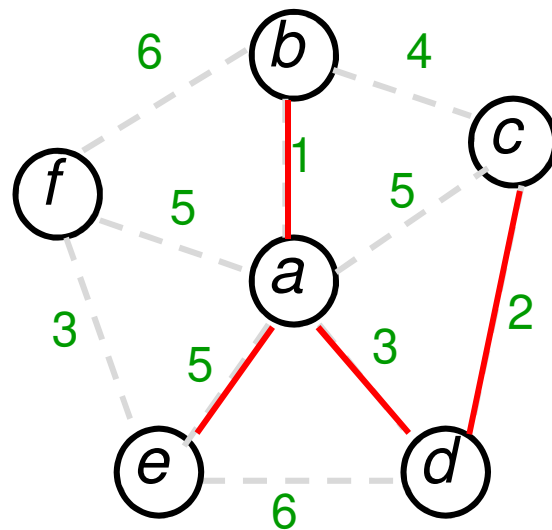
$i \in \bar{S}_T$ adjacent à j

$k := k + 1, \quad \pi_k(i) = \min(\pi_{k-1}(i), c_{ji}),$

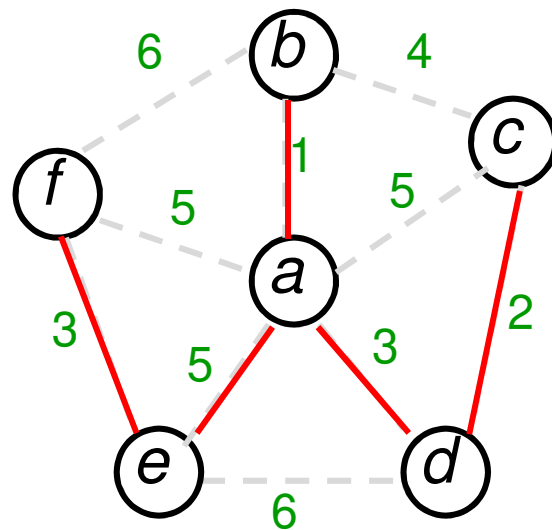
et Retour à 2



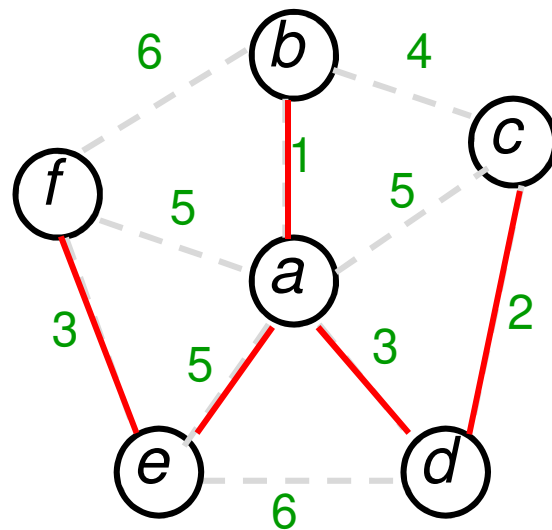
S_T	a	b	c	d	e	f
{ b }	1	0	4	$+\infty$	$+\infty$	6
{b, a }			4	3	5	5
{b,a, d }			2		5	5



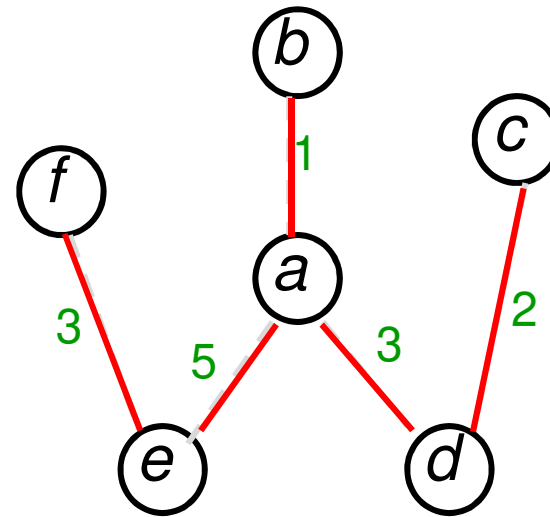
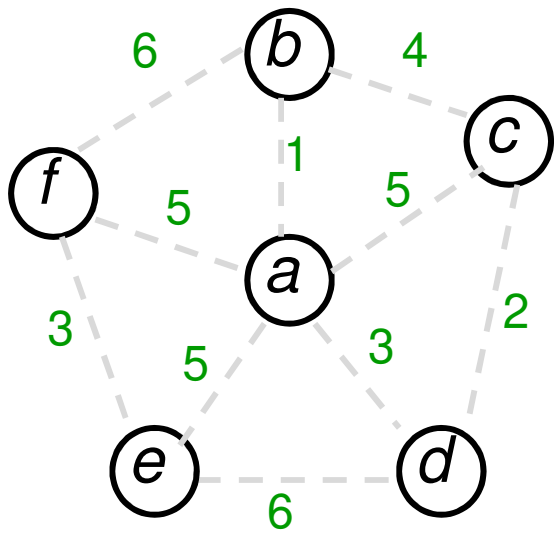
S_T	a	b	c	d	e	f
{ b }	1	0	4	$+\infty$	$+\infty$	6
{b, a }			4	3	5	5
{b,a, d }			2		5	5
{b,a,d, c }					5	5



S_T	a	b	c	d	e	f
{b}	1	0	4	$+\infty$	$+\infty$	6
{b,a}			4	3	5	5
{b,a,d}			2		5	5
{b,a,d,c}					5	5
{b,a,d,c,e}						3



S_T	a	B	c	d	e	f
{b}	1	0	4	$+\infty$	$+\infty$	6
{b,a}			4	3	5	5
{b,a,d}			2		5	5
{b,a,d,c}					5	5
{b,a,d,c,e}						3
{b,a,d,c,e,f} = S						
Fin	1	0	2	3	5	3



$$C_T = 14$$